

DOCUMENTO TÉCNICO MAESTRO · INGENIERÍA DE SOFTWARE

Guía de Metodología Spec-Driven Development (SDD), Arneses de Calidad y Estándares Internacionales

Playbook de Arquitectura, Automatización y Buenas Prácticas para Proyectos Web de Nivel Enterprise y Gubernamental

Proyecto de Referencia: Repositorio Institucional de Planeamientos Curriculares

Institución: Ministerio de Educación Pública (MEP) · DDC **Autores:** Ing. Cristian Vargas & Antigravity (Pair Programming)

Versión: 1.0.1 (Octubre 2026)

1. ¿Qué es Spec-Driven Development (SDD)? (Explicado para Principiantes y Expertos)

En el desarrollo tradicional de software (llamado a menudo *Code-First*), los programadores suelen abrir el editor y comenzar a escribir código inmediatamente según lo que recuerdan de una reunión. Conforme la aplicación crece, surgen inconsistencias en los nombres, rutas rotas, fallos de renderizado y desacuerdos sobre cómo debía funcionar una pantalla.

La Analogía del Arquitecto: Nadie construye un edificio de 20 pisos colocando ladrillos y viendo sobre la marcha dónde poner las puertas. Primero se dibujan los planos estructurales (la especificación). Una vez que los planos están aprobados por ingeniería, los constructores pegan los ladrillos siguiendo fielmente el plano. Si una pared no coincide con el plano, se corrige de inmediato.

Spec-Driven Development (SDD) traslada esta misma disciplina a la ingeniería de software:

- **El Contrato es Primero:** Antes de tocar el código de React o de la API, se redacta el documento de especificación técnica formal (`spec.md`) y el protocolo de calidad (`quality-gates.md`).
- **Única Fuente de Verdad (Single Source of Truth - SSOT):** Todo el equipo (humanos y agentes de IA) consulta el mismo archivo central de especificación. No hay ambigüedades.
- **El Código Existe para Satisfacer la Spec:** El código no inventa reglas; su único trabajo es cumplir la especificación al 100%.
- **Los Tests Verifican el Cumplimiento:** Las pruebas automatizadas leen la especificación y confirman matemáticamente que el sistema la cumple a cabalidad antes de cada commit.

2. El Procedimiento Paso a Paso (El Algoritmo para Nuevos Proyectos)

Para replicar el éxito de este proyecto en futuras iniciativas del MEP o del sector público/privado, se debe seguir estrictamente este flujo de 7 fases:

Fase 1: Spec

Fase 2: Arquitectura

Fase 3: Código

Fase 4: Arneses

Fase 5: Quality Gate

Redactar requerimientos en <code>spec.md</code> y memoria en <code>MEMORY.md</code> .	Definir datos estáticos desacoplados (JSON/APIs) y App Shell.	Componentes puros, modularidad SRP y diseño defensivo.	Escribir pruebas de caos, sensoriales, UI y responsive.	Bloqueo estricto de commit ante fallos de guardrails.
---	---	--	---	---

Detalle de Ejecución de las Fases:

- Fase 1 — Creación del SSOT en `app/docs/`:** Crear los archivos `spec.md` (alcance funcional, filtros, reglas de negocio), `MEMORY.md` (decisiones arquitectónicas y su porqué) y `quality-gates.md` (límites de compilación, umbrales de tamaño y guardrails).
- Fase 2 — Diseño de Arquitectura 100% Estática (JAMstack):** Eliminar cualquier dependencia de lenguajes de servidor en tiempo de ejecución (como scripts PHP lentos o procesos pesados de Node). Diseñar el catálogo maestro como un archivo JSON estructurado (`ddc-planeamientos.json`) que el navegador consulte mediante `fetch()` con parámetros de *cache-busting*.
- Fase 3 — Implementación Modular Limpia (Clean Code):** Desarrollar componentes pequeños con una sola responsabilidad (SRP). Aplicar programación defensiva con operadores de encadenamiento opcional (`?.`) y coalescencia nula (`??`) para que ningún valor inesperado rompa la vista.
- Fase 4 — Construcción de los Arneses de Pruebas:** Construir pruebas unitarias, de paginación, pruebas responsive multiresolución, pruebas de caos (red caída) y pruebas sensoriales de accesibilidad (Ley 7600 y WCAG).
- Fase 5 — Configuración del Pre-Commit Hook:** Instalar un script en `.git/hooks/pre-commit` que ejecute automáticamente los guardrails y los tests antes de permitir cualquier commit. Si una prueba falla, el commit se aborta.
- Fase 6 — Auditoría de Estándares Internacionales:** Validar accesibilidad, semántica HTML5, seguridad OWASP y compatibilidad multiplataforma.
- Fase 7 — Autorización Explícita y Sincronización:** Revisión por el programador senior y ejecución manual del `git push`.

3. Mapa y Catálogo de Archivos del Proyecto (Estructura y Función)

A continuación se detalla cada archivo del repositorio y la función exacta que cumple dentro de la solución:

Archivo / Ruta	Categoría	Propósito y Función Técnica
app/src/App.jsx	COMPONENTE UI	Orquestador raíz de la interfaz. Maneja el estado global del catálogo, filtros, modales, modo claro/oscuro persistente, y aloja el <i>Skip Link</i> accesible para WCAG 2.4.1.
app/src/main.jsx	PUNTO DE ENTRADA	Inicializa el DOM de React 19 y envuelve la aplicación completa dentro del contenedor <code>ErrorBoundary</code> .
app/src/components/GeneralTable.jsx	COMPONENTE UI	Sección principal: catálogo oficial. Integra la barra de búsqueda rápida, selector de filas por página, filtros emergentes por columna (Popovers) y la tabla con paginación matemática.
app/src/components/TopDownloads.jsx	COMPONENTE UI	Muestra los planeamientos más populares descargados por los docentes en tiempo real con barras de progreso y cálculo porcentual.
app/src/components/WelcomeHero.jsx	COMPONENTE UI	Banner institucional con degradado azul MEP, título H1 institucional limpio y descripción normativa de la DDC.
app/src/components/Header.jsx	COMPONENTE UI	Barra superior institucional fija. Contiene logotipos del MEP, contadores globales de planes y descargas, botón de tema claro/oscuro y acceso al modal Acerca de.
app/src/components/Footer.jsx	COMPONENTE UI	Pie de página fijo con badges dinámicos del sistema, enlace normativo de la DDC y badge verde con la vigencia lectiva oficial.
app/src/components/SplashScreen.jsx	COMPONENTE UI	Pantalla de carga inicial institucional. Se muestra mientras el navegador descarga el JSON del catálogo. Maneja contingencia y botón de reintento si no hay red.
app/src/components/ErrorBoundary.jsx	SEGURIDAD UI	Componente de captura de excepciones no controladas de React 19. Evita pantallas blancas y muestra una tarjeta institucional con botón de recarga.
app/src/components/AboutModal.jsx	COMPONENTE UI	Modal institucional que detalla la cobertura curricular de los 197 planes, las 7 ofertas educativas y la versión de compilación.
app/src/components/ResetConfirmModal.jsx	COMPONENTE UI	Modal de confirmación para restablecer contadores de descargas a cero (mantenimiento docente). Soporta cierre con la tecla Escape.
app/src/components/Icons.jsx	ACTIVOS UI	15 iconos SVG nativos optimizados, sin dependencias pesadas de librerías externas de iconos.

Archivo / Ruta	Categoría	Propósito y Función Técnica
app/src/data/planes2027Data.js	CAPA DE DATOS	Módulo desacoplado de consulta HTTP al catálogo mediante <code>fetch()</code> con cache-busting, y persistencia ligera de descargas en <code>localStorage</code> .
app/src/data/version.json	CONFIGURACIÓN	Fuente única de verdad de la versión de la aplicación, título del Hero y badge de vigencia curricular.
app/public/data/ddc-planeamientos.json	CATÁLOGO MAESTRO	Micro-API estática con los 197 registros curriculares normados, rutas web relativas y metadatos de archivos.
app/public/ddc-planeamientos/	ARCHIVOS FÍSICOS	Estructura física de carpetas y 197 archivos comprimidos ZIP en estricto formato POSIX kebab-case.
app/index.html	WEB HOST	Página HTML5 con metadatos W3C, accesibilidad, color temático institucional y tarjetas Open Graph / Twitter para Teams y WhatsApp.
app/docs/spec.md	DOCUMENTACIÓN SDD	Especificación formal completa del sistema, contratos funcionales y casos de uso normados.
app/docs/MEMORY.md	DOCUMENTACIÓN SDD	Registro histórico y bitácora de todas las decisiones arquitectónicas tomadas y sus justificaciones técnicas.
app/docs/quality-gates.md	DOCUMENTACIÓN SDD	Protocolo de las 3 Puertas de Calidad pre-commit y definición de los umbrales de aceptación del software.

4. El Universo de los Arnés de Pruebas (Test Harnesses)

Un **arnés de pruebas** es como la estructura de pruebas donde los ingenieros automotrices estrellan un vehículo contra un muro a 100 km/h para verificar que las bolsas de aire se abran. En software, el arnés somete al código a condiciones extremas para garantizar que nunca falle en manos de los usuarios finales.

En este proyecto implementamos **8 suites automatizadas con 63 pruebas en Vitest**:

Nombre del Arnés	Archivo de Test	Pruebas	Qué Simula y Qué Valida
Arnés Unitario y de Datos	<code>planes2027Data.test.js</code>	7 tests	Validación de carga HTTP vía fetch, inmutabilidad del catálogo original, persistencia de contadores y fusión en tiempo real.
Arnés de Paginación Matemática	<code>GeneralTable.test.jsx</code>	5 tests	Algoritmo de ventana deslizante. Garantiza que en tablas de 197 registros nunca se generen botones de página duplicados ni claves DOM repetidas.
Arnés de Splash Screen	<code>SplashScreen.test.jsx</code>	3 tests	Ciclo de vida del spinner de carga institucional, mensaje de error amigable y botón interactivo de reintento.
Arnés de Guardrails Físicos	<code>guardrails.test.js</code>	10 tests	Auditoría de disco: codificación UTF-8 sin BOM (cero Mojibake), paridad 1:1 entre disco y JSON, y validación estricta de formato kebab-case.
Arnés Responsive (20 Viewports)	<code>responsive.test.jsx</code>	20 tests	Simula 20 pantallas comerciales del mundo real: móviles pequeños (320px, 360px, 375px), tablets (768px, 800px, 820px), laptops (1366px, 1440px), escritorios (1080p) y monitores 4K (3840px).
Arnés de Caos y Resiliencia	<code>chaos.test.jsx</code>	8 tests	Simulación de catástrofes: red completamente caída (Offline), servidores devolviendo HTTP 500 y 502, JSON corrupto, ráfagas de 50 clics simultáneos y cuota de almacenamiento llena (<code>QuotaExceededError</code>).
Arnés Sensorial y Accesibilidad	<code>sensory.test.jsx</code>	8 tests	Cumplimiento de la Ley 7600 de Costa Rica y W3C WCAG 2.1 AA: etiquetas <code>aria-label</code> en todos los botones, contraste de color, navegación por teclado y el <i>Skip Link</i> .
Arnés de Excepciones UI	<code>ErrorBoundary.test.jsx</code>	2 tests	Fuerza el lanzamiento de excepciones imprevistas en componentes React para comprobar que el <code>ErrorBoundary</code> atrapa el error y ofrece contingencia institucional sin pantalla blanca.

5. El Sistema de Guardrails Pre-Commit (Las 3 Puertas de Calidad)

Para que un desarrollador no cometa errores por descuido o fatiga, el sistema implementa un **Quality Gate automatizado** que bloquea la ejecución de `git commit` si no se superan las 3 puertas:

Comando Unificado de Pre-Commit:

```
npm run audit:guardrails && npm test && npm run build
```

Los 10 Guardrails de Seguridad y Disco:

1. **TEST-18: Cero Mojibake UTF-8 (Strict No-BOM):** Escanea todo el código fuente y catálogos en busca de bytes invisibles BOM o caracteres corruptos (ªı , ª©).
2. **TEST-19: Cero Rutas Físicas de Windows:** Rechaza cualquier registro en el JSON que contenga rutas tipo `C:\xampp\...`.
3. **TEST-20: Desacoplamiento del Bundle:** Prohíbe que el archivo de datos JSON sea importado directamente en el JavaScript de React, forzando la carga asíncrona por red.
4. **TEST-21: Lenguaje Institucional Formal:** Verifica que en la interfaz no se utilicen tratamientos informales («tú» o «vos»), exigiendo el tratamiento costarricense formal institucional («usted»).
5. **TEST-22: Nombres Web Seguros (Cero Puntos y Cero Espacios):** Garantiza que ninguna ruta web contenga puntos o espacios que rompan servidores web Linux o Nginx.
6. **TEST-23: Cero Prefijos Numéricos en Textos de Interfaz:** Asegura que los nombres de asignaturas se muestren limpios a los docentes (ej: "Español", no "4. Español").
7. **TEST-24: Verificación Física al 100% de los 197 ZIPs:** Comprueba en el disco duro que cada uno de los 197 archivos existe físicamente y no está vacío.
8. **TEST-28: Guardrail de Directorios Kebab-Case:** Exige que las 241 carpetas en disco estén en minúsculas y separadas exclusivamente por guiones medios (POSIX kebab-case).
9. **TEST-29: Guardrail de Archivos ZIP Kebab-Case:** Exige que los 197 archivos ZIP cumplan la misma regla estricta sin espacios ni caracteres especiales.
10. **TEST-30: Paridad Bidireccional 1:1:** Comprueba matemáticamente que no exista ni un solo archivo huérfano en disco sin registrar en el JSON, ni ningún registro en el JSON sin archivo físico.

6. Matriz de los 25 Estándares Internacionales Cumplidos

Esta matriz es la que certifica que el proyecto posee la misma calidad técnica que una solución creada por una gran transnacional tecnológica:

Estándar Internacional	Organismo Rector	Aplicación Práctica en la Solución DDC
Twelve-Factor App (III - Config)	Cloud-Native Standards	Separación total de configuración y código en <code>version.json</code> y <code>ddc-planeamientos.json</code> . Cero constantes quemadas.
JAMstack Architecture	Modern Web Principles	Arquitectura 100% estática desacoplada. Cero motores lentos en servidor (PHP/Node); micro-API estática mediante <code>fetch()</code> .
App Shell Pattern	W3C / Google PWA	Estructura visual estricta de 100vh con Header y Footer fijos y scroll único en el viewport central.
ISO/IEC 25010 (Portability)	ISO / IEC	Independencia total de plataforma y sistema operativo. 100% rutas relativas funcionales en Linux, Apache, Nginx, Azure o AWS.
IETF RFC 3986	IETF (Internet Standard)	Sintaxis uniforme de URIs en minúsculas kebab-case (<code>^[a-z0-9]+(-[a-z0-9]+)*\$</code>), sin espacios ni diacríticos.
IETF RFC 8259	IETF (JSON Standard)	Intercambio de datos JSON con validación defensiva en el cliente y rechazo inmediato de cargas corruptas o no tabulares.
Unicode RFC 3629	Unicode Consortium	Codificación UTF-8 estricta sin marcas de orden de bytes (BOM) para garantizar inmunidad total contra caracteres corruptos.
IETF RFC 7234	IETF (HTTP Protocol)	Directivas de invalidación de caché (<code>cache: 'no-cache'</code> y parámetro <code>?_t=[timestamp]</code>) para actualizaciones inmediatas.
W3C HTML5 Recommendation	W3C	Uso de etiquetas semánticas nativas: <code><header></code> , <code><main></code> , <code><footer></code> , <code><section></code> , <code><button></code> .
SOLID - SRP	Robert C. Martin	Principio de Responsabilidad Única: separación rigurosa entre vistas, lógica de persistencia, iconografía y filtros.
Defensive Programming	SEI / Carnegie Mellon	Uso sistemático de encadenamiento opcional (<code>?.</code>) y operadores de coalescencia nula (<code>??</code>) contra excepciones de nulos.
State Immutability Pattern	React Core Standards	Estado borrador (Draft State) en menús de filtrado: congela la tabla y evita re-renders innecesarios hasta presionar Aplicar.
Deterministic Algorithms	IEEE Computer Society	Paginación matemática basada en ventanas deslizantes estrictas con claves DOM únicas que impiden duplicados.
W3C WCAG 2.1 AA (1.1.1)	W3C WAI / Ley 7600 CR	Alternativas textuales accesibles: 100% de los botones e iconos cuentan con <code>aria-label</code> o <code>title</code> descriptivo.
W3C WCAG 2.1 AA (1.4.3)	W3C WAI	Contraste cromático estricto superior a 4.5:1 en texto regular y 3:1 en títulos grandes en modo claro y modo oscuro.
W3C WCAG 2.1 AA (2.1.1)	W3C WAI	Operabilidad completa mediante teclado: tabulación fluida y anillos de foco visibles (<code>focus:ring-2</code>).
W3C WCAG 2.1 AA (2.4.1)	W3C WAI	Implementación de <i>Skip Link</i> al inicio del documento para saltar directamente al catálogo curricular con la tecla Tab.

Estándar Internacional	Organismo Rector	Aplicación Práctica en la Solución DDC
WAI-ARIA Dialog Pattern 1.2	W3C WAI	Modales accesibles con atributo <code>role="dialog"</code> , <code>aria-modal="true"</code> y cierre automático al presionar Escape.
Unicode CLDR / i18n	Unicode Consortium	Ordenamiento alfabético sensible al idioma español mediante <code>localeCompare(..., 'es')</code> respetando diacríticos de la RAE.
Google Core Web Vitals (LCP)	Google Web Team	Largest Contentful Paint óptimo gracias al Splash Screen inmediato y a un bundle comprimido de apenas 83 KB en gzip.
Google Core Web Vitals (CLS)	Google Web Team	Cumulative Layout Shift de cero absoluto: alturas mínimas reservadas (<code>min-h-[480px]</code>) que previenen brincos de pantalla.
Google Core Web Vitals (INP)	Google Web Team	Interaction to Next Paint ultra reactivo: búsqueda predictiva instantánea y descargas escalonadas a 350ms sin congelar la UI.
OWASP Client-Side Security	OWASP Foundation	Almacenamiento seguro en <code>localStorage</code> limitado a un mapa de contadores numéricos; cero tokens o scripts expuestos.
Web Storage Quota Defense	W3C Web Storage	Captura preventiva de la excepción <code>QuotaExceededError</code> para garantizar que la app siga funcionando en memoria ante discos llenos.
OWASP A03 (XSS Mitigation)	OWASP Foundation	Cero uso de <code>dangerouslySetInnerHTML</code> ; React escapa nativamente cualquier inyección maliciosa de código HTML/JS.

7. Checklist para el Próximo Proyecto (El Template de Arranque de Ing. Cristian Vargas)

Para iniciar el próximo proyecto con el mismo nivel de excelencia, siga este checklist paso a paso:

Paso 1: Definir la Memoria y la Especificación (Día 1)

Crear la carpeta `docs/` y redactar `spec.md` (alcance funcional) y `MEMORY.md` (decisiones arquitectónicas).

Paso 2: Establecer la Fuente Única de Verdad (SSOT)

Centralizar textos, versiones y títulos en archivos JSON desacoplados (como `version.json`), nunca quemados en el código.

Paso 3: Instalar el Arnés de Pruebas Multi-Capa

Configurar Vitest con Testing Library y crear suites para: 1) Capa de datos, 2) Responsive en 20 resoluciones, 3) Caos/Red, 4) Accesibilidad WCAG, y 5) Guardrails de disco.

Paso 4: Blindar el Repositorio con el Pre-Commit Hook

Configurar el Quality Gate en `.git/hooks/pre-commit` para obligar la ejecución limpia de guardrails, tests y compilación antes de cada commit.

Paso 5: Respetar la Regla de Control de Versiones

Trabajar en ramas locales, validar el 100% de pruebas en verde, generar la compilación de producción (< 300 KB) y sincronizar a GitHub únicamente con aprobación expresa.

Ministerio de Educación Pública de Costa Rica · Dirección de Desarrollo Curricular

Documento elaborado en colaboración de Pair Programming por Ing. Cristian Vargas & Antigravity AI.

© 2026 Todos los derechos reservados.